KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
School of Computer Engineering

Software Engineering

Software Requirements Specification (Extended)

ColdStore-Vision

Automated Cold-Storage Sack Inspection from Video

Submitted by
[Name] - [Roll number]
[Name] - [Roll number]
[Name] - [Roll number]

Group: [Group name] **Section:** [Section]

Submitted to: [Faculty name]

Version 1.0 · 2 November 2026

How this document is organised

Part A is the SRS, in IEEE 830 order. It states what the system must do and treats the system as a black box. Part B applies the rest of the syllabus to this project; designs and plans are kept there because they do not belong in an SRS. Part C maps each syllabus topic to its section.

Part A: Software Requirements Specification

- A.1 Introduction
- A.2 Overall description
- A.3 Specific requirements
- A.4 Constraints, assumptions, dependencies
- A.5 Validation of this SRS

Part B: Syllabus extension

- B.1 Software engineering basics and process model
- B.2 Software design
- B.3 Project management
- B.4 Coding and testing
- B.5 Quality, reliability and maintenance
- B.6 Emerging topics

Part C: Syllabus traceability

Part A: Software Requirements Specification

A.1 Introduction

A.1.1 Purpose

This document specifies the requirements of ColdStore-Vision, a software system that compares two video walkthroughs of the same cold-storage stack and reports the condition of each sack. It is written for the project team, the course evaluators, testers and any future customer.

A.1.2 Scope

Cold stores hold thousands of onion, potato and garlic sacks. Checking them by hand for rot, settling or missing stock is slow and irregular. ColdStore-Vision takes a Day 1 recording and a Day 2 recording, and gives every sack one status with a written reason.

In scope: offline processing of recordings; detection, matching and diagnosis of sacks; reports and videos; tools to train the detector.

Out of scope in this version: the inspection drone (concept only), live video streams, integration with stock or dispatch software, automatic stack-level stock counts, and a web service.

A.1.3 Definitions

Term	Meaning
Sack	One jute or mesh bag of produce.
Bay, tier, slot	Horizontal section of a rack, its row counted from the bottom, and a position within the row.
Keyframe	A sharp frame chosen from the video for analysis.
Homography	An 8-parameter mapping between two views of a flat surface.
Parallax	Near objects shifting more than far ones when the camera moves.
GLCM	Grey-level co-occurrence matrix, a measure of texture.
mAP@50	Mean average precision of the detector at 50% overlap.
UNVERIFIED	Status meaning the system could not compare the sack reliably.

A.1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- R. Mall, *Fundamentals of Software Engineering*, PHI Learning.
- ColdStore-Vision project guide (GUIDE.md) and master specification.

A.1.5 Stakeholders

Stakeholder	Need from the system
Cold-store manager	A quick view of which sacks need attention, and where they are
Inspector or operator	A simple way to record and run an inspection
Dispatch and inward staff	Missing and new sacks to reconcile against their registers
ML engineer or developer	A way to retrain the detector and change thresholds
Course evaluators	Evidence of the method and honest results

A.1.6 Requirements gathering

Activity	How it was applied
Study of existing documents	Read the project's master specification, then the documentation of Polycam exports and YOLO dataset formats.
Interview	Discussions with the project owner fixed the key decisions: no site markers, train on Kaggle, compare onion stack only.
Observation	Two visits to a real godown on 21 and 22 October 2026 to record it. This showed dim light, pillars and touching sacks.
Task analysis	Inspection was split into: record, align, find sacks, match, diagnose, report.
Scenario analysis	For one sack: unchanged, removed, swapped, rotting, settled, hidden by a pillar, cut by the frame, not filmed on Day 2.
Form analysis	Output fields were derived from what staff need: the inward register, the dispatch log and an action per sack.

A.1.7 Requirements analysis: problems found and resolved

Type	Problem	Resolution
Anomaly	"Report a sack as missing" has several meanings. A sack hidden by a pillar also looks missing.	Measurable rule in FR-07.1, and a separate UNVERIFIED status in FR-07.7.
Anomaly	"Sack has changed colour" had no threshold.	Relative hue above 15 degrees, measured after lighting compensation (FR-07.4, FR-05).
Inconsistency	The master spec required training to finish first. The owner wanted all modules built in parallel.	Owner's decision adopted. A fallback detector and a synthetic test detector allowed parallel work.
Inconsistency	The spec preferred LiDAR capture, but the actual LiDAR recording was a room scan, not a rack pass.	Video is the default. LiDAR is optional through --prefer-lidar (FR-01.2).
Incompleteness	No status existed for sacks the system could not compare.	UNVERIFIED added (FR-07.7).
Incompleteness	A swapped sack was not covered by the original statuses.	"Possibly replaced" rule added (FR-07.6).
Incompleteness	Nothing said how to treat sacks cut off at the frame edge.	Truncation rule added (FR-07.8).

A.2 Overall description

A.2.1 Product perspective

ColdStore-Vision is a self-contained command-line tool. It depends on a phone camera for input and on Kaggle only for training. It does not form part of a larger system today.

A.2.2 Product functions

1. Read recordings in several formats.
2. Choose sharp keyframes.
3. Align the two days and build panoramas.
4. Outline every sack.
5. Compensate for lighting.

6. Match each sack across days.
7. Diagnose each sack and give a reason.
8. Locate it by bay, tier and slot.
9. Write five outputs.

A.2.3 User characteristics

User	Technical skill	Frequency of use
Inspector	Can use a phone and copy a command; no vision knowledge	Each inspection
Store manager	Can open a web page; no command-line use	After each inspection
ML engineer	Python and machine learning	When retraining or tuning

A.2.4 Operating environment

Windows 11, Python 3.12, CPU only, at least 8 GB of RAM and about 1 GB of disk for models. A modern browser opens the reports. Training uses a Kaggle NVIDIA T4 GPU.

A.2.5 External interfaces

Interface	Description
User interface	Command line for running; offline HTML pages and MP4 videos for results.
Input files	Video (.mp4, .mov, .m4v, .avi); Polycam zip with frames, camera JSON and depth; JPEG or PNG images.
Config file	YAML: configs/pipeline_config.yaml.
Output files	MP4, HTML, JSON, JPEG.
Hardware	Any phone recording at 1080p or 4K, 30 fps. LiDAR is optional.
Software	OpenCV, PyTorch, ultralytics, kornia, OpenCLIP, SciPy, scikit-image; Kaggle CLI for training.

A.2.6 Use cases

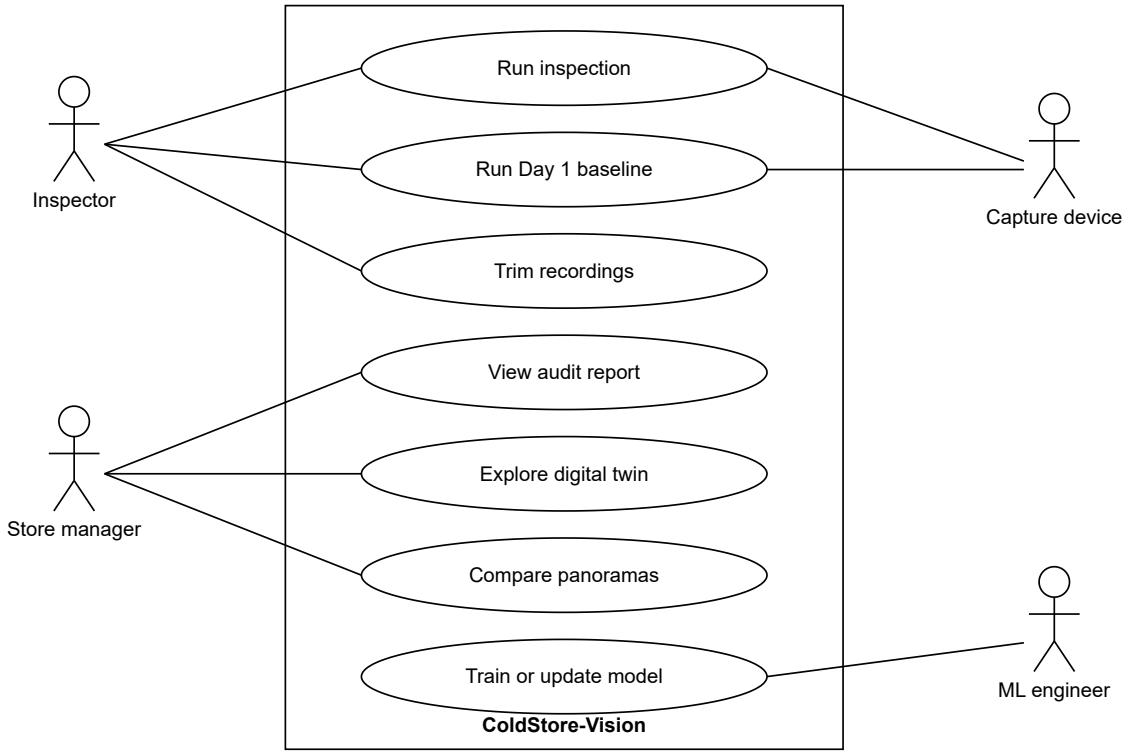


Figure 1: Use case diagram

Field	UC-1: Run inspection
Actor	Inspector
Precondition	Two recordings of the same stack exist; the model file is installed.
Main flow	1. Inspector runs process.py with the two paths. 2. System processes both. 3. System writes the outputs and prints the folder.
Alternative 1	Day 2 omitted: system writes a Day 1 baseline.
Alternative 2	Views too different: affected sacks are UNVERIFIED and the inspection still finishes.
Exception	Path does not exist: system prints "input not found" and stops.
Postcondition	A new run folder holds all outputs.

A.3 Specific requirements

A.3.1 Functional requirements

All requirements below are implemented. **Known limit L1 (FR-07.1):** when a removed sack is overlapped by a neighbour, the synthetic real-photo test gives WARNING instead of MISSING.

FR-01 Input handling

ID	Requirement	Verified by
FR-01.1	The system shall accept a video file (.mp4, .mov, .m4v, .avi), a Polycam Raw Data export (.zip or folder), or a folder of at least 5 images as the recording for each day.	test_polycam_roundtrip; onion run
FR-01.2	When a folder holds both a video and a Polycam export, the system shall use the video unless the user passes --prefer-lidar.	Code inspection

ID	Requirement	Verified by
FR-01.3	The system shall accept start and end times in seconds to process part of a recording.	Onion run (trim 9.8 s)
FR-01.4	If Day 2 is omitted, the system shall produce a Day 1 baseline report with the sack count and a recording quality check.	outputs/day1_baseline
FR-01.5	If an input path does not exist, the system shall stop and print the path with the message "input not found".	TC-B2

FR-02 Keyframe selection

ID	Requirement	Verified by
FR-02.1	The system shall resize all frames to a width of 1280 pixels before analysis.	test_keyframes_and_registration
FR-02.2	The system shall select a keyframe every 0.5 m of camera travel when poses exist, and every 0.5 s otherwise.	test_keyframes_and_registration
FR-02.3	The system shall replace a keyframe whose Laplacian variance is below 100 with the sharpest frame within 5 frames either side.	Code inspection, path testing (B.4)

FR-03 Alignment between days

ID	Requirement	Verified by
FR-03.1	The system shall pair each Day 1 keyframe with the Day 2 keyframe showing the same scene, using a pair only when it has at least 40 inlier matches (LoFTR mode).	test_keyframes_and_registration; onion run (10 pairs)
FR-03.2	The system shall compute a mapping that warps the Day 2 frame onto the Day 1 frame, and shall reject an implausible mapping.	test_homography_recovers_known_warp
FR-03.3	The system shall build one panorama per day, aligned to each other pixel for pixel.	test_mosaics_aligned

FR-04 Sack detection

ID	Requirement	Verified by
FR-04.1	The system shall output an outline, box and confidence for each sack, rack beam and empty slot at a confidence of at least 0.25.	test_sack_count_matches_day1
FR-04.2	If the trained model file is missing, the system shall log a warning and use the fallback detector.	Code inspection

FR-05 Lighting compensation

ID	Requirement	Verified by
FR-05.1	The system shall remap Day 2 colours so its histogram matches Day 1, excluding sacks that are missing or new.	test_histogram_matching_undoes_global_lighting
FR-05.2	The system shall report each sack's hue change relative to the hue drift of the background.	test_global_hue_shift_on_bimodal_background

FR-06 Sack matching

ID	Requirement	Verified by
FR-06.1	The system shall assign Day 1 sacks to Day 2 sacks one to one, and reject a pair whose cost exceeds 0.6.	test_hungarian_iou_matching
FR-06.2	The system shall re-align each matched pair locally before comparing them.	Onion run
FR-06.3	When a sack appears in several keyframe pairs, the system shall report the majority status.	Onion run

FR-07 Diagnosis

ID	Requirement	Verified by
FR-07.1	The system shall mark a sack MISSING when at least 45% of its eroded area changes (20% if no Day 2 sack covers the spot) and no Day 2 sack matches it, or when LiDAR depth grows by more than 0.30 m.	test_every_anomaly_detected...; test_missing_uses_depth
FR-07.2	The system shall mark a sack NEW when a Day 2 sack has no Day 1 match.	test_every_anomaly_detected...
FR-07.3	The system shall mark a sack WARNING when its rot-coloured area grows by 3 percentage points, and CRITICAL at 8.	test_necrosis_and_glcm_respond_to_rot
FR-07.4	The system shall mark a sack WARNING on relative hue change above 15 degrees, GLCM contrast rise above 35%, or homogeneity drop above 20%.	test_necrosis_and_glcm_respond_to_rot
FR-07.5	The system shall mark a sack WARNING on settling above 4 px and CRITICAL above 10 px, capped at WARNING when the views differ.	test_flow_detects_downward_shift
FR-07.6	The system shall flag a sack as possibly replaced when hue shifts by at least 10 degrees and at least 40% of its area differs.	Onion run (Sack_011)
FR-07.7	The system shall mark a sack UNVERIFIED when fewer than 50% of the sacks in its view match, or its hue jumps by more than 90 degrees (object in front).	Onion run (13 sacks)
FR-07.8	The system shall not mark a sack MISSING if it is cut by the left or right frame edge, or smaller than 1% of the frame.	Code inspection

FR-08 Location indexing

ID	Requirement	Verified by
FR-08.1	The system shall assign each sack a bay, tier and slot without markers or a bay configuration file.	test_bay_indexer_uses_uprights_and_rows; test_bays_and_tiers

FR-09 Outputs

ID	Requirement	Verified by
FR-09.1	The system shall write a Day 2 video with each sack outlined in its status colour.	test_outputs_written
FR-09.2	The system shall write an audit report (HTML and JSON) listing, per sack: slot, status, measurements, reason and action.	test_outputs_written

ID	Requirement	Verified by
FR-09.3	The system shall write a side-by-side video of Day 1 and aligned Day 2 with the reason for each flag.	test_outputs_written
FR-09.4	The system shall write a bay-by-tier digital twin page showing before and after crops for a clicked sack.	test_outputs_written
FR-09.5	The system shall write a panorama page with a Day 1 / Day 2 curtain slider.	test_outputs_written
FR-09.6	The system shall write run_info.json with inputs, trims, notes, keyframe counts, KPIs and run time.	Onion run

FR-10 Model training support

ID	Requirement	Verified by
FR-10.1	The system shall provide commands to scrape, filter, auto-label, split and validate a sack image dataset.	Dataset v1.1 built
FR-10.2	The system shall provide a command that trains the detector on a cloud GPU and fetches the weights and metrics.	Kaggle T4 run, 105 epochs

A.3.2 Non-functional requirements

ID	Attribute	Requirement	Measured result	Status
NFR-P1	Performance	A 15-second recording pair shall complete within 20 minutes on a laptop CPU with no GPU.	791 s (13.2 min) for the onion pair. About 20 min for full-length 4K walks.	Met
NFR-A1	Accuracy	Detector mask mAP@50 shall be at least 0.85 on the test set.	0.797	Not met
NFR-A2	Accuracy	Detector mask precision shall be at least 0.80 on the test set.	0.798	Not met (marginal)
NFR-A3	Accuracy	On the synthetic aisle, every planted anomaly shall be found with no false alarm.	All found, no false alarm	Met
NFR-R1	Reliability	All automated tests shall pass before a release.	16 of 16 passed on 2 Nov 2026	Met
NFR-R2	Reliability	A view that cannot be compared shall produce UNVERIFIED, never MISSING or CRITICAL.	Enforced by the pair gate (FR-07.7)	Met
NFR-U1	Usability	An inspection shall need one command with at most two required paths.	process.py --day1 X --day2 Y	Met
NFR-U2	Usability	Every flagged sack shall state its reason in a plain sentence.	Reason column in the audit report	Met
NFR-M1	Maintainability	All thresholds shall be changeable in one YAML file without code changes.	configs/pipeline_config.yaml	Met
NFR-M2	Maintainability	A new detector shall be installable by replacing one weights file.	models/weights/best.pt	Met
NFR-O1	Portability	The system shall run on Windows 11 with Python 3.12 on CPU.	All runs on this setup. Linux and macOS not tested.	Met (Windows only)

ID	Attribute	Requirement	Measured result	Status
NFR-C1	Compatibility	HTML outputs shall open offline in a modern browser, as single files.	Images embedded as base64 JPEG	Met
NFR-C2	Compatibility	Output videos shall play in a desktop media player.	mp4v by default. Browser playback needs avc1 (openh264).	Partly met
NFR-S1	Security	Inspection footage and reports shall remain on the local machine.	No inspection step uploads data. Only training images go to Kaggle.	Met
NFR-S2	Security	API tokens shall not be stored in the project folder.	Kaggle token is in the user profile folder	Met
NFR-B1	Backup	Each run shall write to a new folder, so earlier results are not overwritten.	outputs/run_<timestamp>/ by default	Met
NFR-L1	Scalability	The system shall handle at least 40 sacks in one frame.	44 sacks separated in one Day 1 frame. Whole-aisle runs not tested.	Met for one stack

Availability: a figure such as 99.9% uptime does not apply, because this version is a batch tool, not a running service. It would apply to the SaaS form in B.6.

A.4 Constraints, assumptions and dependencies

Constraints

- No GPU on the inspection machine, so processing is CPU based.
- Free cloud GPU quota limits training runs.
- No site markers or bay configuration may be required.
- Training images could only come from the web at the start, so the first model was trained without cold-store frames.

Assumptions

- Both recordings cover the same stack, walking at about 0.5 m/s.
- Sacks are visible from the aisle; sacks behind the front layer cannot be assessed.
- Most of the frame does not change between days, so lighting can be matched across the frame.
- Visual change is a useful sign of spoilage. Rot inside a sack with no surface change cannot be seen.

Dependencies

- Python packages listed in requirements.txt.
- Model downloads on first run (OpenCLIP, LoFTR).
- Kaggle service for training.

A.5 Validation of this SRS

Property of a good SRS	How this document meets it
Concise and unambiguous	"Shall" statements with numeric thresholds, such as 15 degrees or 0.30 m.
States what, not how	Part A describes behaviour. Algorithms and designs are in Part B. Thresholds are observable behaviour.
Easy to change	Numbered groups; thresholds also sit in one config file.

Property of a good SRS	How this document meets it
Consistent	The conflicts in A.1.7 were resolved with the project owner.
Complete	Scenario analysis covered each sack condition. Known limit L1 is stated.
Traceable	Every FR names a test or a run. Part C links the syllabus to sections.
Verifiable	"User-friendly" was replaced by NFR-U1 and NFR-U2, which can be checked.

Part B: Syllabus Extension

B.1 Software engineering basics and process model

Software, its characteristics and components

Software differs from a manufactured product. It is developed rather than built on a line, it does not wear out, and it can be changed after delivery. It also has more parts than a program does:

Component	In this project
Programs	src/, process.py, tools/
Data	Dataset v1.1 (152 images, 2,782 sack outlines), trained weights
Documents	README, GUIDE.md, this SRS, project report
Operating procedures	Recording guidelines, retraining steps

Role of a software engineer: understanding requirements, choosing techniques, testing, estimating, managing risk and documenting, not only coding.

Software crisis and emergence of software engineering

The software crisis described projects that ran late, over budget and unreliable as size grew past what one person could hold in mind. Engineering discipline arose in response, through methods, abstraction and measured quality.

Similarities to and differences from conventional engineering

Aspect	Conventional engineering	This project
Shared	Requirements, design, testing, quality targets	The same phases apply
Main cost	Manufacturing	Development; copies are free
Failure	Parts wear out	Faults are present from the start, or appear when the input changes, such as a new viewpoint
Change	Costly after build	Cheap to edit, but side effects are hard to see
Standard parts	Catalogue components	Libraries such as OpenCV and ultralytics

Quality attributes

Those that matter most here are correctness (NFR-A), reliability (NFR-R), usability (NFR-U), maintainability (NFR-M), portability (NFR-O) and efficiency (NFR-P). See A.3.2 for measured values.

Process models compared

Model	Essential idea	Fit for this project
Classical waterfall	Sequential phases with no going back	Poor. Requirements changed after seeing real footage.
Iterative waterfall	Waterfall with feedback to earlier phases	Partial. Feedback was needed, but also early working versions.
Prototyping	Build a throwaway model to clarify requirements	Useful early. The synthetic aisle and fallback detector acted as a prototype.

Model	Essential idea	Fit for this project
Evolutionary	Deliver a core, then grow it in increments	**Best fit, and the model followed.**
Spiral	Risk-driven loops of plan, evaluate, build	Its risk focus was used, but the full model is heavy for a small team.
RAD	Fast construction with reusable parts and user feedback	Partly matched through library reuse, but needs a customer on hand and a time-boxed team.
Agile	Short iterations, working software, changing requirements welcome	Close in spirit, but no formal sprints were held.

Evolutionary increments followed

1. **Core:** all modules, tested on synthetic data with a test detector.
2. **Detector:** dataset v0, then v1.1, trained model installed.
3. **Real footage:** Day 1 baseline, then Day 2. SIFT pairing failed across days, so LoFTR replaced it.
4. **Safety rules:** occlusion, truncation, majority vote and replaced-sack rules, each driven by a false result seen in real footage.
5. **Planned:** fine-tune model v2 on cold-store frames.

Agile, XP and Scrum

Agile versus other models: plan-driven models fix requirements early and measure progress by documents. Agile expects change and measures progress by working software. **Extreme Programming** pushes good practices to the limit: pair programming, test-driven work, continuous integration, simple design. Here, automated tests and simple design were used; working with an AI assistant resembled pairing. **Scrum** organises work into sprints with a backlog, daily stand-up and review, using a product owner and Scrum master. It was not used formally, but it would suit the next phase, since a drone-based product will keep changing requirements.

B.2 Software design

Design process and what makes a good design

Design turns the SRS into modules, their interfaces, data structures and algorithms. A good design is correct, understandable, efficient and easy to maintain. The design rests on layering (ingestion, registration, segmentation, analytics, reporting) and one configuration file.

Cohesion and coupling

Good modules have high cohesion and low coupling.

Module	Cohesion	Reason
optical_flow.py	Functional	Every function serves one task: measure motion
lighting_normalizer.py	Functional	All of it serves lighting compensation
anomaly_engine.py	Functional / sequential	Measurements flow into one diagnosis
pipeline.py	Sequential / communicational	Steps pass results along and share the Inspection data
dashboard_builder.py	Communicational	Different reports built from the same inspection data
tools/dataset/common.py	Logical	Shared paths and CSV helpers; acceptable for utilities

Module pair	Coupling	Reason
pipeline to SackDetector	Data	Passes an image, gets a list of Instance. Replacing the model touches nothing else.
tracker to anomaly_engine	None direct	pipeline.py passes results between them
All modules to config	Stamped	They receive the config dictionary but read only their own section; a small weakness accepted for simplicity
reporting to pipeline	Stamped	Reporting reads the Inspection structure, so changes to it may affect reports

Function-oriented design: DFD

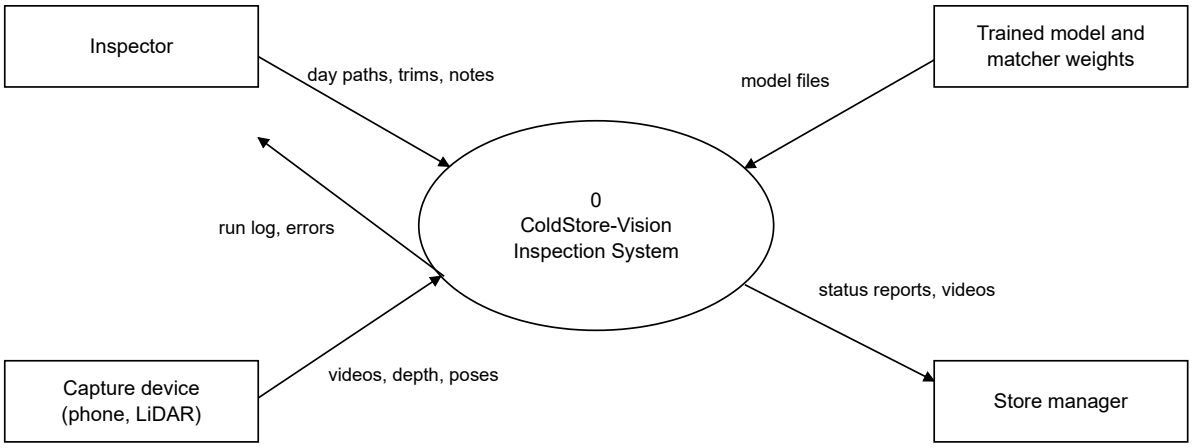


Figure 2: Level 0 DFD (context diagram)

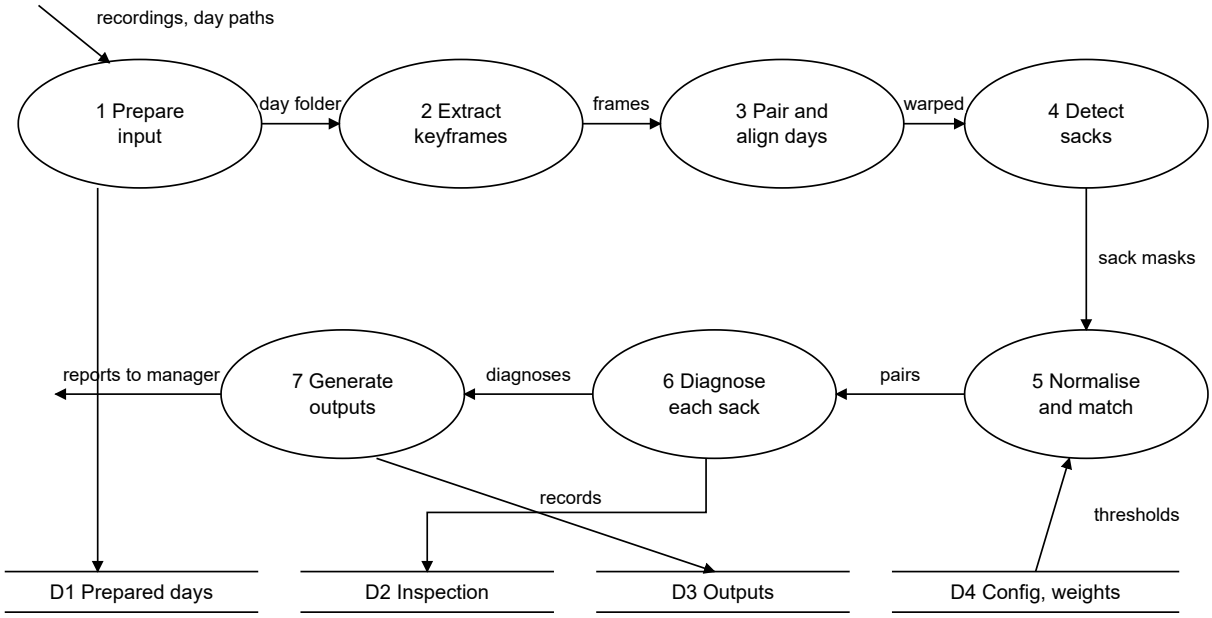


Figure 3: Level 1 DFD

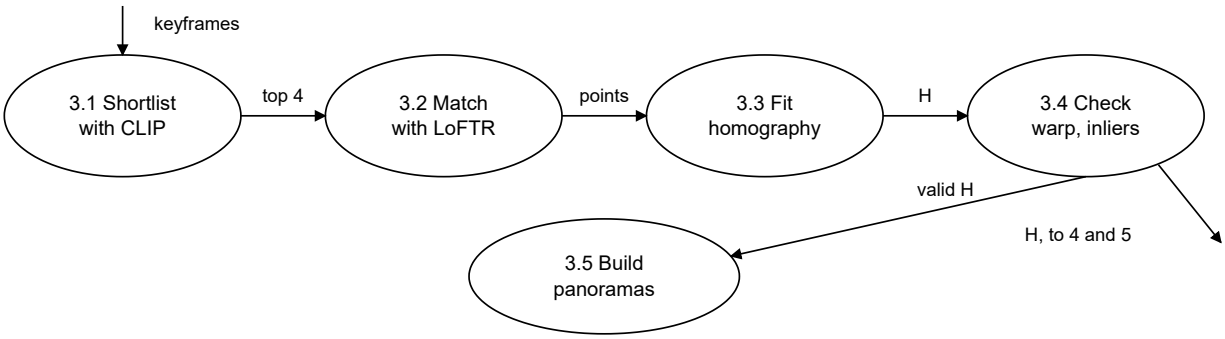


Figure 4: Level 2 DFD, process 3 (pair and align days)

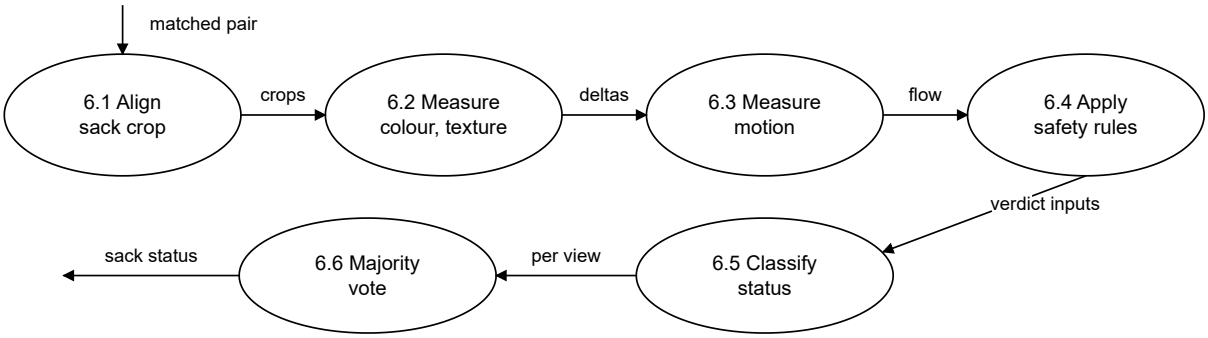


Figure 5: Level 2 DFD, process 6 (diagnose each sack)

Data dictionary

Data item	Definition
recording	video file | Polycam export | image folder
keyframe	index + time + image + [pose] + [depth] + sharpness
H	3 × 3 homography matrix + inlier count + ok flag
sack mask	class + confidence + box + binary mask + polygon
diagnosis	status + hue + rot fraction + GLCM + flow + reason + action
status	NORMAL | WARNING | CRITICAL | MISSING | NEW | UNVERIFIED
reports	overlay video + audit report + side-by-side video + digital twin + panorama slider

Structural design

Transform analysis applied to the Level 1 DFD gives this structure chart: input handling, the central transform in run_inspection, and output generation. It reads left to right, with the data returned to run_inspection written beside each module.

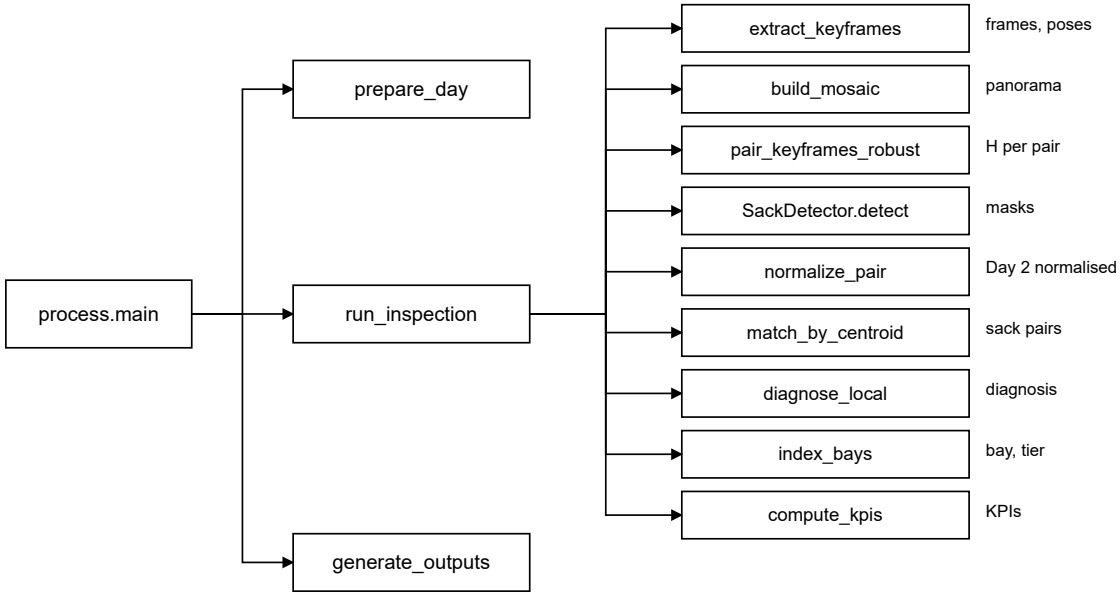


Figure 6: Structure chart

Object-oriented design

Concepts used: *abstraction and encapsulation* in SackDetector, which hides the choice of YOLO or YOLOE; *composition* in Inspection, which holds keyframes, pairs and sack records; *polymorphism* by duck typing, since SyntheticColorDetector and SackDetector offer the same detect() method. Inheritance is not used, as data classes and composition were sufficient.

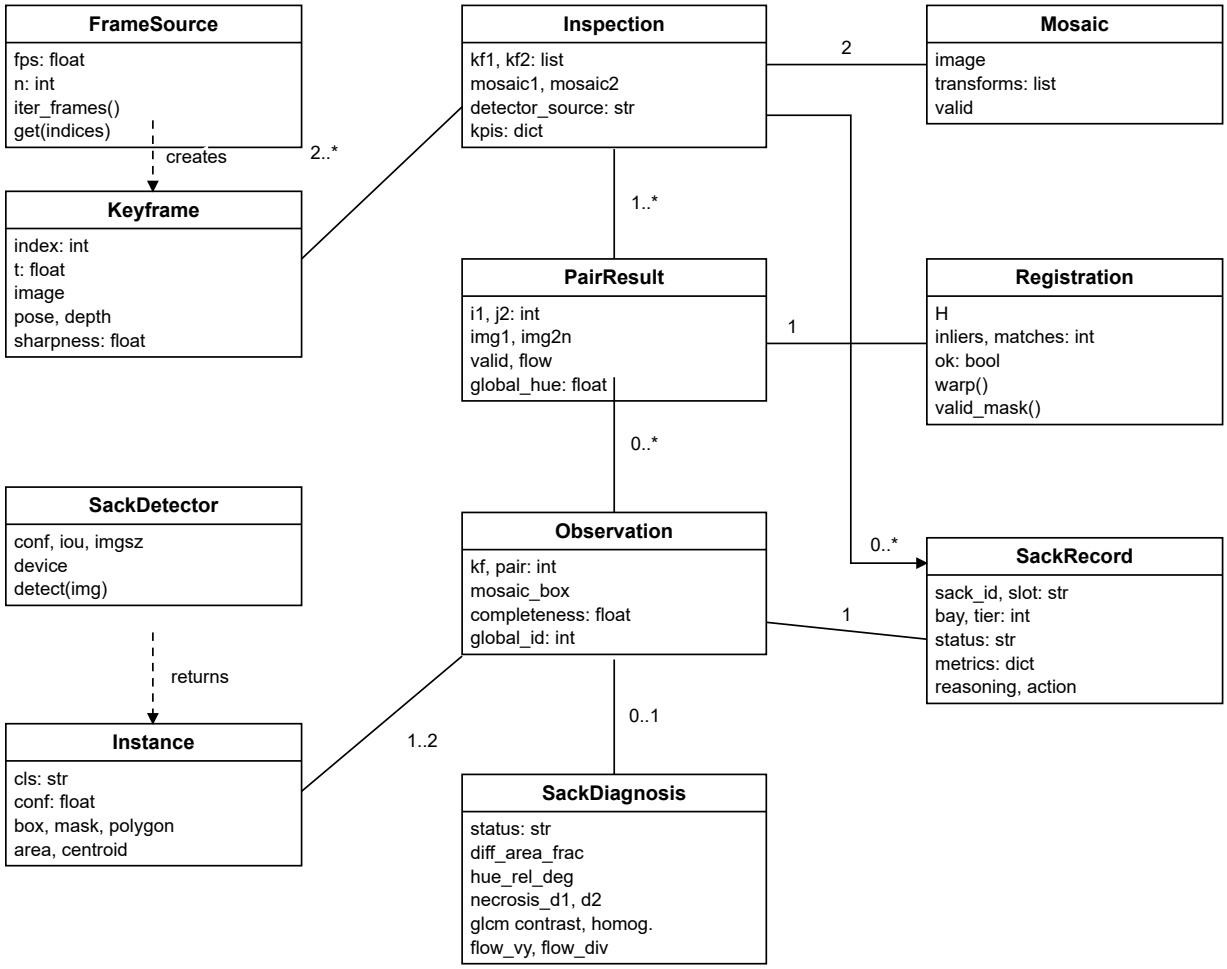


Figure 7: Class diagram (main fields shown)

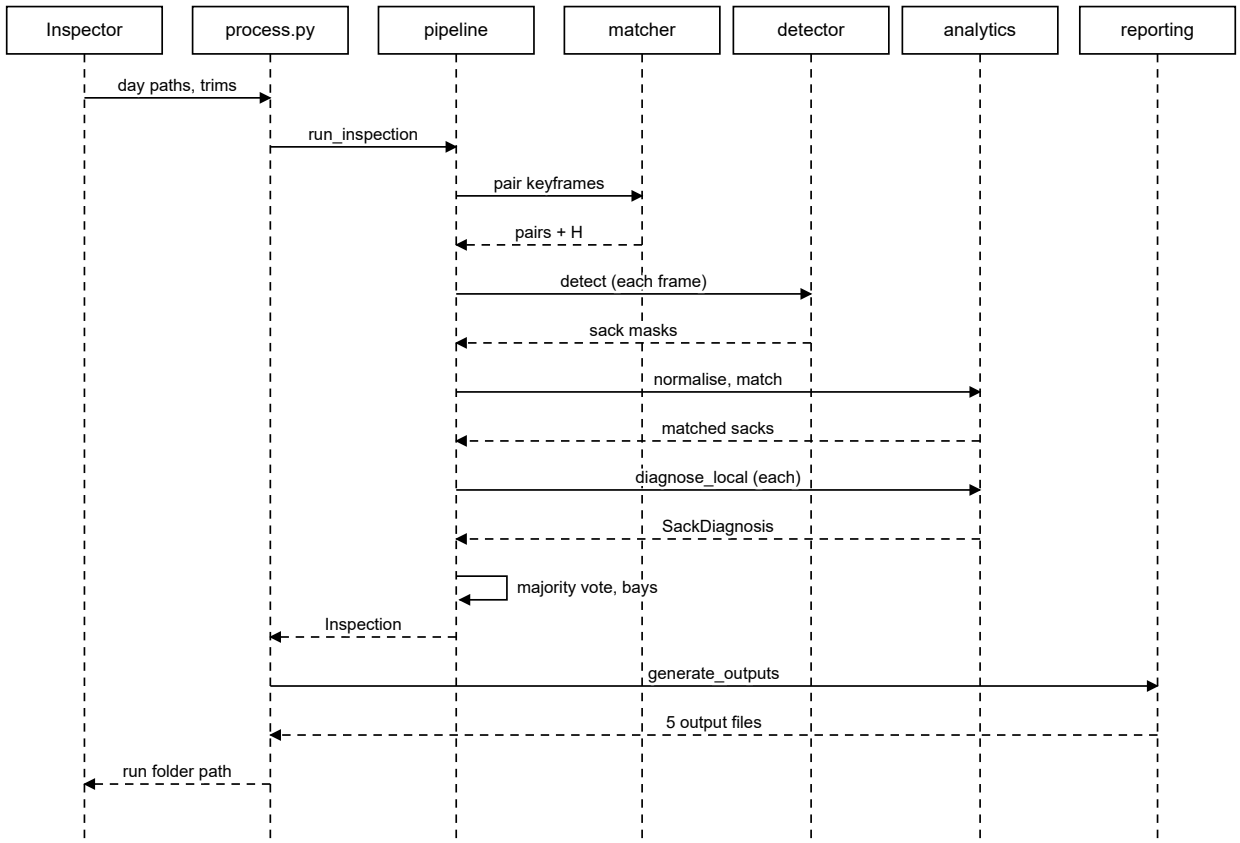


Figure 8: Sequence diagram for Run inspection

B.3 Project management

Why software projects are complex to manage

Software is invisible, requirements change, and each project differs. Here the main uncertainties were real footage quality, cloud GPU quota and whether vision methods would survive the change between days.

Project planning

Planning covers size, effort, schedule, staffing, risks and quality. The estimates below use the measured code size.

Size: lines of code

Part	**LOC**
src/ (inspection pipeline)	1902
process.py + main.py (entry points)	235
tools/dataset (dataset building)	1261
tools/remote + models/train_yolo.py (training)	419
tools/synth (synthetic test data)	311
Delivered product	**4128**
Not counted: tests/	198
Not counted: video/ (explainer, not part of the product)	878

Counted as non-blank, non-comment Python lines.

Size: function points

Type	Count	Weight (average)	Points	Items
External inputs	5	4	20	Day 1 recording, Day 2 recording, Polycam export, CLI options and notes, YAML config
External outputs	7	5	35	Five inspection outputs, Day 1 baseline, run_info.json
External inquiries	3	4	12	Audit table filter, click-a-sack in digital twin, curtain slider
Internal logical files	3	10	30	Prepared day folders, inspection records, label dataset
External interface files	3	7	21	YOLO weights, CLIP and LoFTR models, Kaggle kernel results
UFP			**118**	

The 14 general system characteristics, rated 0 to 5: Data communications 1, Distributed processing 1, Performance 4, Heavily used configuration 1, Transaction rate 1, Online data entry 1, End-user efficiency 3, Online update 0, Complex processing 5, Reusability 4, Installation ease 3, Operational ease 3, Multiple sites 2, Facilitate change 4. Their sum is DI = 33.

$$TCF = 0.65 + 0.01 \times 33 = 0.98 \quad FP = 118 \times 0.98 = 115.6$$

Empirical estimation: expert judgement and Delphi

In **expert judgement**, one expert estimates from experience; the risk is personal bias. In **Delphi**, several experts estimate anonymously through a coordinator, see a summary and revise, repeating until the numbers settle. The table shows the method using illustrative figures, not an actual survey; replace them if your group runs the exercise.

Estimator	Round 1 (KLOC)	Round 2 (KLOC)	Round 3 (KLOC)
A	2.5	3.5	4.0
B	6.0	5.0	4.5
C	4.0	4.0	4.0
Mean	4.2	4.2	4.2

Heuristic estimation: COCOMO

Basic COCOMO, with size = 4.128 KLOC. Effort $E = a \times KLOC^{b}$ person-months, time $T = 2.5 \times E^{c}$ months.

Mode	Effort (PM)	Time (months)	Avg. staff
Organic	$2.4 \times 4.128^{1.05} = 10.6$	$2.5 \times 10.6^{0.38} = 6.1$	1.7
Semi-detached	$3.0 \times 4.128^{1.12} = 14.7$	$2.5 \times 14.7^{0.35} = 6.4$	2.3
Embedded	$3.6 \times 4.128^{1.2} = 19.7$	$2.5 \times 19.7^{0.32} = 6.5$	3.0

Organic mode applies: small size, small team, and limited outside constraints. **Intermediate COCOMO** adjusts it with cost drivers (others nominal at 1.00):

Driver	Rating	Multiplier	Reason
RELY: required reliability	High	1.15	A false alarm costs a manual stack check
CPLX: product complexity	High	1.15	Multi-stage computer vision with learned models

Driver	Rating	Multiplier	Reason
AEXP: application experience	Low	1.13	First cold-store vision system for the team
MODP: modern practices	High	0.91	Automated tests, config-driven thresholds
TOOL: software tools	High	0.91	OpenCV, ultralytics, Kaggle GPU, AI coding assistant

$$EAF = 1.238 \quad E = 3.2 \times 4.128^{1.05} \times 1.238 = 17.5\ PM \quad T = 2.5 \times 17.5^{0.38} = 7.4\ months$$

Comparison with actual: the working system took about one week of elapsed time (19 to 26 October 2026), far below 7.4 months. COCOMO does not account for an AI coding assistant or heavy library reuse, so it reflects a conventional team.

Work breakdown structure

1. **Requirements:** study spec; decide scope; write requirements.
2. **Dataset:** scrape; filter and deduplicate; auto-label; review and fix; split and validate.
3. **Model:** set up Kaggle; train; evaluate; install weights.
4. **Pipeline:** ingestion; registration; segmentation wrapper; lighting; matching; diagnosis; bay index.
5. **Outputs:** videos; audit report; digital twin; panorama slider.
6. **Testing:** synthetic data; unit tests; integration and system tests.
7. **Field work:** record Day 1; record Day 2; run, tune and re-run.
8. **Documentation:** guide; explainer video; SRS; report.

Activity network and critical path (CPM)

Durations are ideal working days, in a re-baselined plan. Each node shows activity (duration) and ES-EF / LS-LF. Shaded nodes are critical.

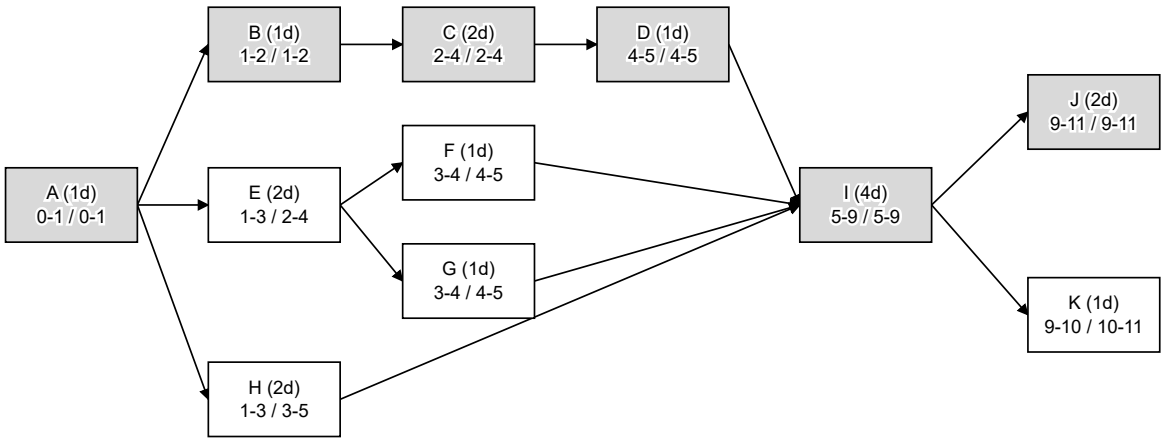


Figure 9: Activity network

Activity	Name	Dur.	Pred.	ES	EF	LS	LF	Slack
A	Requirements and spec analysis	1	-	0	1	0	1	0
B	Scrape and filter images	1	A	1	2	1	2	0
C	Auto-label and review	2	B	2	4	2	4	0
D	Train YOLO on Kaggle	1	C	4	5	4	5	0
E	Core pipeline modules	2	A	1	3	2	4	1

Activity	Name	Dur.	Pred.	ES	EF	LS	LF	Slack
F	Report and video outputs	1	E	3	4	4	5	1
G	Synthetic test suite	1	E	3	4	4	5	1
H	Capture Day 1 and Day 2 footage	2	A	1	3	3	5	2
I	Integrate, tune on real footage	4	D, F, G, H	5	9	5	9	0
J	Explainer video	2	I	9	11	9	11	0
K	Guide and documents	1	I	9	10	10	11	1

Critical path: A - B - C - D - I - J, length 11 days.

PERT

Expected time t = (o + 4m + p) / 6 and variance = ((p - o) / 6)^2. Values for o and p are the team's judgement.

Activity	o	m	p	t	Variance
A	0.5	1	2	1.08	0.062
B	0.5	1	2	1.08	0.062
C	1	2	4	2.17	0.250
D	0.5	1	3	1.25	0.174
E	1	2	3	2.00	0.111
F	0.5	1	2	1.08	0.062
G	0.5	1	2	1.08	0.062
H	2	2	3	2.17	0.028
I	2	4	7	4.17	0.694
J	1	2	3	2.00	0.111
K	0.5	1	2	1.08	0.062

On the critical path: expected length = 11.75 days, variance = 1.354, standard deviation = 1.16 days. There is about an 84% chance of finishing within 12.9 days (mean plus one standard deviation).

Gantt chart

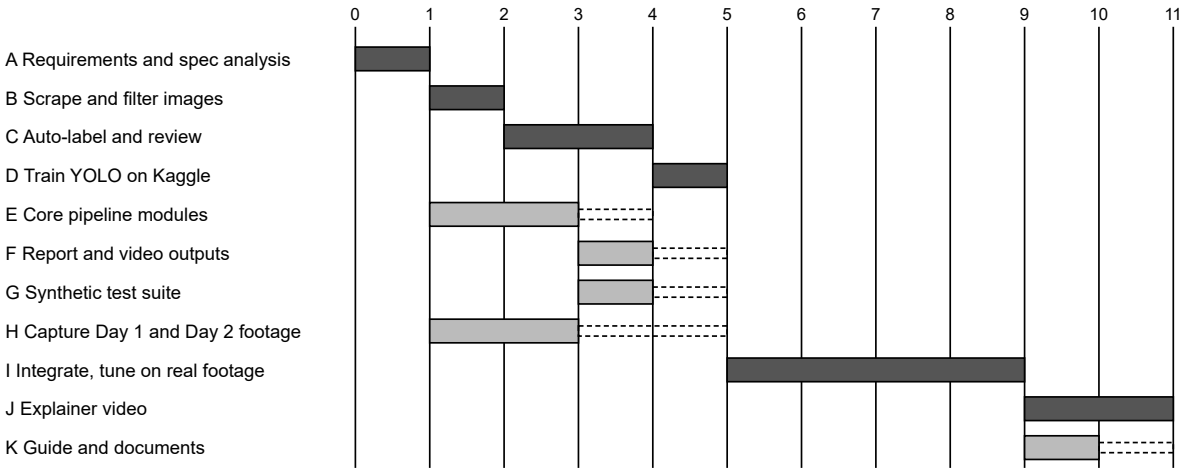


Figure 10: Gantt chart in project days. Dark bars are critical; dashed bars show slack.

Actual milestones

Date (2026)	Milestone
19 Oct	Scope decisions; dataset work started
20 Oct	Model v1 trained on Kaggle T4 (105 epochs)
21 Oct	Day 1 recorded; baseline run
22 Oct	Day 2 recorded
26 Oct	Onion comparison delivered; explainer video and guide completed
2 Nov	SRS and project report completed; all 16 tests re-run

Organisation and team structure

Common structures are **chief programmer** (one senior leads, suits small and simple work), **democratic** (decisions shared, suits hard problems but small groups only) and **mixed** (for large projects). A democratic team suits this project because the hard problems benefit from discussion. Fill in who did what:

Role	Responsibility	Member
Project lead and requirements	Scope, SRS, site visits	[to fill]
ML engineer	Dataset and model training	[to fill]
Vision developer	Pipeline modules	[to fill]
Test and documentation	Tests, report, video	[to fill]

Risk management

Risks are identified, assessed (probability × impact on a 1 to 5 scale), then avoided, transferred or reduced. The risks below actually arose, except where the last column says otherwise.

Risk	P	I	Exposure	Action taken	Outcome
Footage from different angles on each day	5	5	25	Stronger matcher; UNVERIFIED status; recording guide	Occurred. 13 of 22 sacks unverified
Web images differ from godown frames	4	4	16	Grey and blur augmentation; v2 fine-tune planned	Occurred. Slightly under accuracy target
SIFT cannot pair days	4	5	20	Fallback: CLIP + LoFTR	Occurred and resolved
False alarms from lighting or a pillar	4	4	16	Histogram matching; occlusion rule	Occurred and reduced
Cloud training ran on CPU unnoticed	3	4	12	Force T4 accelerator in the script	Occurred and resolved
Model training blocks other work	3	3	9	Build in parallel with a fallback and test detector	Avoided
Manual labelling takes too long	4	3	12	Agent review with SAM 2.1 fixes	Avoided
New threshold breaks earlier behaviour	3	4	12	Regression tests	Occurred and caught
Data loss	2	4	8	New folder per run; no version control backup yet	Open: add a git repository

B.4 Coding and testing

Coding standards and code review

The code follows Python conventions: modules per task, typed function signatures, docstrings explaining why, and thresholds in config. **Code review** comes in two forms: a *walkthrough*, where members trace code by hand, and an *inspection*, which checks against common errors. Reading the code against its imports while writing the guide found five wrong library entries in the draft documentation. A formal, recorded inspection has not been held.

Basic concepts

Term	Meaning	Example here
Error / fault	A mistake made in code	No rule for an object blocking the view of a sack
Failure	Observable wrong behaviour	Sacks behind a pillar reported MISSING
Test case	Input, conditions and expected result	See the tables below
Test suite	A set of test cases	tests/ (16 cases)

Objective of testing: to find faults, not to prove there are none. **Verification** asks "are we building the product right?" and checks each phase's output: reviews, unit tests. **Validation** asks "are we building the right product?" and checks the final product against needs: the real godown run. **Testing activities:** plan, design cases, execute, check results, debug, then re-test.

Unit, integration and system test results

Run with pytest on 2 November 2026: **16 passed in 93.56 s.**

Test case	Level	Input	Expected	Result
test_homography_recovers_known_warp	Unit	Known warp	Registration recovers the warp	Pass
test_histogram_matching_undoes_global_lighting	Unit	Globally darkened image	Lighting difference removed	Pass
test_global_hue_shift_on_bimodal_background	Unit	Two-colour background, shifted hue	Drift measured correctly	Pass
test_necrosis_and_glcm_respond_to_rot	Unit	Sack with rot patch	Rot fraction and GLCM both change	Pass
test_erosion_shrinks_mask	Unit	Binary mask	Eroded mask is smaller	Pass
test_flow_detects_downward_shift	Unit	Image shifted down	Positive vertical flow	Pass
test_hungarian_iou_matching	Unit	Two sets of boxes	Correct one-to-one pairs	Pass
test_bay_indexer_uses_uprights_and_rows	Unit	Beam and sack boxes	Correct bay and tier	Pass
test_keyframes_and_registration	Integration	Synthetic aisle	Keyframes paired	Pass
test_sack_count_matches_day1	Integration	Synthetic aisle	Count equals known sacks	Pass

Test case	Level	Input	Expected	Result
test_every_anomaly_detected_without_false_alarms	System	Planted anomalies	All found, no false alarm	Pass
test_missing_uses_depth	Integration	Synthetic depth	Depth drives MISSING	Pass
test_bays_and_tiers	Integration	Synthetic aisle	Correct layout	Pass
test_mosaics_aligned	Integration	Two days	Panoramas line up	Pass
test_outputs_written	System	Inspection result	All five outputs exist	Pass
test_polycam_roundtrip	Integration	Synthetic Polycam export	Frames, poses, depth preserved	Pass

Test drivers and stubs

Driver: pytest with the fixtures in tests/conftest.py, which call modules and check results. **Stub:** SyntheticColorDetector stands in for the YOLO model. It offers the same detect() method, finding sacks by colour in synthetic images, which let the pipeline be tested before any model existed.

Top-down and bottom-up testing

Bottom-up tests small modules first with drivers; **top-down** tests the main control flow first with stubs. This project used both, which is **mixed (sandwich)** integration: unit tests for low-level modules, and run_inspection early with the stub detector. Integration then proceeded incrementally rather than all at once.

Black box (functional) testing

Cases come from the requirements, through equivalence classes and boundary values. Status shows what was actually run; the rest form the plan.

ID	Class or boundary	Input	Expected	Status
TC-B1	Valid: video	IMG_3866.MOV + IMG_3939.MOV	Five outputs	Pass (onion run)
TC-B2	Invalid: no such path	--day1 no_such_path_for_test	"input not found", stops	Pass (2 Nov)
TC-B3	Valid: Day 2 omitted	--day1 only	Baseline report	Pass (21 Oct)
TC-B4	Valid: Polycam export	Synthetic export	Frames, poses, depth converted	Pass (automated)
TC-B5	Boundary: 4 images	Folder of 4 images	Rejected	Planned
TC-B6	Boundary: 5 images	Folder of 5 images	Accepted	Planned
TC-B7	Boundary: inliers 39 and 40	Pair at each count	Rejected, then accepted	Planned
TC-B8	Boundary: sharpness 99.9 and 100	Frame at each score	Replaced, then kept	Planned
TC-B9	Boundary: rot gain 2.9, 3, 8 points	Sack at each gain	NORMAL, WARNING, CRITICAL	Planned

White box (structural) testing

Example: the blur rejection loop in keyframe_extractor.py.

```
for c in cands:                                  # D1
    if sharp[c] < thr:                           # D2
        lo, hi = max(0, c - win), min(n, c + win + 1)
        c = lo + int(np.argmax(sharp[lo:hi]))
    if not chosen or c != chosen[-1]:            # D3, D4 (two conditions)
        chosen.append(c)
```

Cyclomatic complexity V(G) = decisions + 1 = 4 + 1 = 5, so five independent paths need testing:

Path	Condition	Test idea
1	cands is empty	Skip loop; chosen stays empty
2	Sharp frame, first in list	Added directly
3	Sharp frame, differs from the last chosen	Added
4	Blurred frame, replacement differs	Replaced, then added
5	Blurred frame, replacement equals the last chosen	Duplicate dropped

Performance testing

Measured on a laptop CPU: 791 s for the onion pair (11 Day 1 and 15 Day 2 keyframes, 10 pairs). Full-length 4K walks take about 20 minutes.

System testing

Alpha testing was the team's run on real godown footage. **Beta testing** at a cold store with real staff has not been done. Results from the real run: 22 sacks, 4 NORMAL, 4 WARNING, 1 MISSING and 13 UNVERIFIED, with a swapped sack found.

Acceptance testing

Criterion	Result
A pair of phone videos gives all five outputs with one command	Accepted
A changed sack is flagged with a reason	Accepted (Sack_011)
Blocked sacks are not called missing	Accepted (pillar case)
Most sacks get a definite verdict from handheld footage	**Not accepted:** 13 of 22 unverified; needs consistent capture
Detector reaches mask mAP@50 of 0.85	**Not accepted:** 0.797

Regression testing

The test suite was re-run after each rule change. A real example: adding the occupancy rules made removed-sack tests fail, which led to the truncation rule and a more realistic gap fill in tools/synth/make_real_pair.py.

B.5 Quality, reliability and maintenance

Software quality assurance plan

Item	Plan
Purpose	Ensure that releases meet Part A
Standards	Python conventions; IEEE 830 for requirements
Reviews	SRS review by team and owner; code walkthrough before each release

Item	Plan
Testing	All automated tests pass; one real-footage run per release
Metrics	mAP, precision, share of UNVERIFIED sacks, false alarms per run, run time
Problem reporting	Record each false result with sack ID, reason text and run folder (see GUIDE.md checklist)
Configuration management	To add: git repository, tagged versions of code, dataset and weights. Dataset zips are already versioned (v0, v1, v1.1).

ISO 9000

ISO 9000 is a family of standards for quality management systems. ISO 9001 applies to organisations that design, develop and maintain software. It certifies the process rather than the product, and requires documented procedures, reviews and corrective action. This project has parts of it: documented requirements, test records and corrective action.

SEI Capability Maturity Model

Level	Name	Key trait	This project
1	Initial	Ad hoc; depends on individuals	Started here
2	Repeatable	Basic project management; size, cost, schedule tracked	**Partly reached:** estimates, milestones and a risk table exist, with repeatable commands
3	Defined	Standard documented process	Partly: pipeline and dataset steps are documented; no organisation-wide process
4	Managed	Process and product measured	Product measured (mAP, tests), process not
5	Optimising	Continuous improvement from the data	Not reached

ISO 9000 asks whether a documented process exists, while CMM gives steps for improving one.

Software reliability

Reliability is the probability of failure-free operation over a period. Common metrics are ROCOF, MTTF, MTTR, MTBF and POFOD. Reliability here depends strongly on the input: the same code does well on consistent footage and poorly on changed viewpoints, a known feature of software reliability, which depends on the operational profile. Too few real runs exist to estimate MTTF. Available measures:

- **Per verdict:** detector precision of 0.798 and a synthetic false-alarm count of 0.
- **Design for reliability:** the UNVERIFIED status turns unreliable conditions into a declared limit rather than a wrong answer.

Software maintenance

Type	Meaning	Example
Corrective	Fix faults	Kaggle multi-GPU AutoBatch crash; pillar false MISSING
Adaptive	Fit a new environment	Support 4K and Polycam input; future drone camera
Perfective	Improve or add	Trim options; replaced-sack rule; model v2
Preventive	Ease future change	Config-driven thresholds; test suite

Thresholds need retuning for each new site, and the model needs retraining when conditions change.

Reverse engineering

Reverse engineering recovers design from existing code. GUIDE.md section 5 was produced this way: file roles and libraries were recovered by reading the code and its imports.

Re-engineering

Re-engineering improves a working system without changing what it does for the user. Here the pairing step was re-engineered: SIFT was replaced by CLIP and LoFTR behind the same interface, returning the same Registration structure. A config switch keeps both (registration.matcher).

Software reuse

Reused item	Source	Benefit
Computer vision primitives	OpenCV	SIFT, Farneback, warping, without writing them
Detector architecture and training	ultralytics YOLO11s-seg	Transfer learning from pretrained weights
Dense matching	kornia LoFTR	Solved cross-day pairing
Image embeddings	OpenCLIP	Retrieval and dataset filtering
Assignment and texture	SciPy, scikit-image	Hungarian algorithm, GLCM
Within the project	tools/dataset, detector seam	Autolabel filters shared with fix tools; pipeline reused by process.py and main.py

B.6 Emerging topics

Software as a Service

In SaaS, the provider hosts one application and customers use it over the internet, usually by subscription. The current version is a local tool, but SaaS fits its future well: cold stores have no GPUs or ML staff.

Aspect	Today (local tool)	As SaaS
Compute	Customer's laptop	Provider's GPU servers
Updates	Manual copy of weights	Central; all customers improve at once
Interface	Command line	Web upload and dashboard
New requirements	None	Login, roles, tenant data separation, encryption, billing, 99.9% availability
Risk	Local data only	Footage leaves the site, so privacy terms are needed

AI-based software engineering

AI now supports the software process itself. This project used it in three ways, recorded here so the method is transparent:

- **AI coding assistant:** Claude Code was used to write and revise code, scripts and documents under human direction. The team set scope, gave footage and checked results.
- **AI agents in the data pipeline:** reviewer agents judged images and label quality, and gave correction points for SAM 2.1. This reduced an estimated 8 to 15 hours of manual labelling.
- **AI components in the product:** YOLO, LoFTR and CLIP. Their behaviour is learned, not written, so they are tested statistically (mAP) rather than only with fixed cases.

AI output must be verified: tests caught regressions, and checks against the code found documentation errors.

Part C: Syllabus Traceability

Module	Topic	Section
1	Software and its characteristics; components; role of software engineer	B.1
1	Software crisis; emergence of SE; comparison with conventional engineering	B.1
1	Quality attributes	B.1, A.3.2
1	Waterfall, iterative waterfall, prototyping, evolutionary, spiral, RAD	B.1
1	Agile: essential idea, Agile versus others, XP, Scrum	B.1
2	Requirements gathering and analysis (anomaly, inconsistency, incompleteness)	A.1.6, A.1.7
2	SRS document and IEEE 830	Part A
2	Design process; good design; cohesion and coupling	B.2
2	Function-oriented design: DFD levels, structural analysis, structure chart	B.2
2	Object-oriented design: concepts; use case, class, sequence diagrams	B.2, A.2.6
3	Project management complexity; project planning	B.3
3	Size metrics: LOC, function points	B.3
3	Empirical estimation: expert judgement, Delphi	B.3
3	Heuristic estimation: COCOMO	B.3
3	WBS, activity network, CPM, PERT, Gantt	B.3
3	Organisation and team structure; risk management	B.3
4	Code review	B.4
4	Testing basics, objectives, verification and validation, testing activity	B.4
4	Unit, functional and performance testing; top-down and bottom-up; drivers and stubs	B.4
4	Black box, white box, integration, system, acceptance, regression testing	B.4
5	SQA plan; ISO 9000; SEI-CMM	B.5
5	Software reliability	B.5
5	Maintenance; reverse engineering; re-engineering; reuse	B.5
Emerging	Software as a Service; AI-based software engineering	B.6